

Evora.ai: AI-Powered Product Video Generation Platform

Evora.ai aims to let users upload a product image and text prompt to automatically generate a polished marketing video. This involves combining state-of-the-art AI video models with a robust software stack and user-friendly interface. Below, we survey existing platforms, recommend AI models and APIs, and outline the backend and frontend architecture for such a system.

Existing AI Video Platforms & Capabilities

Recent AI video tools (Synthesia, Runway/Luma, Kling, etc.) show what's possible. Synthesia's SaaS can turn text into AI-generated videos with realistic avatars and voices ¹. Dozens of new image-to-video platforms are emerging: in 2025 it's now common for "marketing: turning product shots into scroll-stopping ads" or "product demos from a single image" ². In general, image-to-video AI uses generative models (diffusion, GANs, transformers) to infer backgrounds, motions, and transitions from a still picture ³ ⁴. For example, OpenAI's Sora model can convert text or images into high-fidelity clips (multi-shot with consistent characters) ⁵ ⁶. Luma (Runway) Dream Machine focuses on ultra-fast text-to-video with camera-motion controls ⁷, while Kling AI offers a unified pipeline where text/image inputs yield multi-shot stories with native audio and storyboard editing ⁸. Platforms like DomoAI also allow users to set multiple keyframes (start/end frames, up to 8) and let the AI interpolate smooth transitions ⁹. These examples suggest that Evora.ai could leverage similar scene-control features (e.g. keyframe/storyboarding) to guide the AI output.

Models and APIs for Image-to-Video Generation

The core is choosing ML models or APIs. Today the leading technology is **video diffusion models**. OpenAI's Sora 2 Pro is state-of-the-art: it "transforms still images into cinematic, physics-aware videos with synchronized audio" in one API call ⁶. Users upload a product photo and describe the action (e.g. "rotate the product 360°, add a camera pan") – Sora 2 Pro then generates up to 20 seconds of 720p/1080p video, complete with realistic physics (objects obey gravity, collide properly) and matching sound effects or music ¹⁰ ¹¹. In blind tests it scored very high for realism and prompt accuracy. Crucially, Sora 2 Pro explicitly supports *product marketing*: "Transform static product photos into dynamic showcase videos... dramatically reducing production costs" ¹². (For example, the frame below – generated by Sora – shows two robotic figures animated into an action scene from a static image.)



Advanced image-to-video models (e.g. OpenAI's Sora 2 Pro) can animate a single product image into a dynamic video with realistic motion and synchronized sound ⁶ ¹³ .

Because models like Sora 2 Pro may be costly or access-restricted, open-source alternatives can be used. **Stable Video Diffusion** (by Stability AI) is available to run in-house: it's a diffusion pipeline that takes one image and generates a short clip ¹⁴ . It can run via Hugging Face or local GPUs (using the `diffusers` library). However, current open models produce shorter videos (~1–4 seconds at 576×1024) and may lack perfect realism ¹⁴ . These can still be useful for prototyping or effects (e.g. rotating an object). Other emerging models/pipelines include Runway's **Stable Video Diffusion**, and community tools like CogVideo or PikaLabs that can be chained together. Specialized APIs (via WaveSpeed, Runway, etc.) expose many of these: for instance, WaveSpeed's Sora 2 Pro API wraps the OpenAI model with per-second pricing ¹⁵ .

Besides video, we'll need speech and audio. Text-to-speech (TTS) APIs are mature: ElevenLabs, AWS Polly, Google WaveNet, or even Sora 2's built-in dubbing can generate narration from script. Synthesia itself supports 140+ voices and languages ¹⁶ , so Evora should likewise allow voiceover selection (multiple languages, accents). We can feed the user's script (or a script generated by an LLM like GPT-4) into a TTS engine to get a voice track. Similarly, background music and sound effects can be chosen from a library or generated (some models like Sora auto-generate ambient sound if prompted ¹⁷).

In summary, the AI pipeline will likely combine: (a) a **scene-generation model** (e.g. Sora 2 Pro via API or open-source video diffusion) to create each shot; (b) a **language model** (e.g. GPT-4 or Llama) to expand a brief user prompt into a detailed script/storyboard with multiple scenes; (c) **text-to-speech models** to voice the narration; and (d) traditional tools (e.g. FFmpeg or MoviePy) to stitch clips, add transitions, overlays, and output the final MP4. This multi-stage approach mimics workflows in advanced platforms (Synthesia, Kling).

Backend Architecture: Languages, Frameworks & Databases

For the backend, we recommend a **polyglot microservices** design. Use **Python** (FastAPI or Flask) for the AI pipeline: model orchestration, data preprocessing, and communicating with ML libraries (PyTorch). Use **Node.js (Express)** or Go for the API layer and real-time client communication. This hybrid is common in AI

products: Python handles heavy ML tasks (batches of inference on GPUs), while Node/Go handles high-concurrency APIs and asynchronous I/O ¹⁸. They communicate via REST/gRPC or a message queue (e.g. Redis, RabbitMQ, or AWS SQS). Dockerize each service for deployment.

For data storage, a mix of databases is ideal. User accounts, projects, and metadata can live in a relational DB (PostgreSQL or MySQL). However, Evora's video project data (scene definitions, script text, etc.) are naturally JSON-like and may change shape. As Synthesia experienced, **MongoDB** or another document DB can be a good fit: "Storing video documents as JSON, MongoDB offered the flexibility and performance" they needed ¹⁹. Alternatively, a single Postgres DB with JSONB columns could suffice. At minimum, use a fast cloud NoSQL for schemaless parts (video configurations) and SQL for user/auth. Also use a **vector database** (e.g. Pinecone, Weaviate) if you plan semantic search of assets or prompts.

Media files (uploaded images, interim results, final videos) should be stored on a scalable object store. For example, AWS S3 or Google Cloud Storage. Synthesia uses S3 for large data ²⁰. Serving finished videos to users is done via a CDN (e.g. CloudFront). Use Redis or Memcached to cache frequent reads (e.g. model embeddings, small static content, or even small images).

For the software stack: React (or Vue/Angular) is a strong choice for the frontend, since you've already built a proof-of-concept UI. On the back end, Node.js with Express (or NestJS) works well for REST APIs, or you could use Python's FastAPI for everything. Key is asynchronous support: calls to ML workers should not block the API. Use Celery (Python) or BullMQ (Node) to queue long-running tasks (video generation steps), with the workers running on GPU instances. Docker and Kubernetes (EKS) are advisable for deployment. In fact, **Synthesia's production uses Kubernetes (Amazon EKS) and AWS Batch to orchestrate hundreds of GPU instances** ²⁰. They "batch process datasets... often running up to 100 instances at the same time" to train models ²¹. We can similarly spin up GPU-powered pods (e.g. AWS P4/P5 instances) for model inference.

Security and scalability: use OAuth or JWT for auth, HTTPS for all traffic. Store secrets in a vault. CI/CD pipelines (GitHub Actions, Jenkins) should build Docker images and deploy to Kubernetes (with Helm or similar). Monitor with Prometheus/Alertmanager or a cloud service.

AI Pipeline Design and Scalability

An end-to-end pipeline might look like this:

1. **User Request:** Frontend uploads image & prompt to backend via REST API.
2. **Job Queuing:** Backend enqueues a video-generation job (e.g. in Redis/RabbitMQ).
3. **Script/Storyboard Generation:** A Python worker calls an LLM (GPT) to generate a multi-shot storyboard or script from the prompt (e.g. "Shot 1: product on turntable; Shot 2: product in use by person...").
4. **Scene Generation:** For each shot, the worker invokes the image-to-video model (e.g. Sora 2 Pro API or Stable Diffusion). For Sora 2 Pro, send the image + shot description to get a short clip (4–20s) ⁶. If using an open model, run PyTorch diffusion pipelines on a GPU (possibly splitting long videos into segments).
5. **Voice Synthesis:** Simultaneously or next, feed the script text to a TTS API/engine to generate audio clips.

6. **Post-Processing:** The worker uses ffmpeg to combine all clips and audio, adding transitions, captions, and final encoding.
7. **Result Storage:** The final video (MP4) is uploaded to S3. Metadata (URL, status) is updated in the database.
8. **Notification:** The backend notifies the frontend (via WebSocket or polling) that the video is ready; the user can preview/download.

Each of these steps should be handled by scalable workers. For instance, multiple GPU pods can process different scenes or different user jobs in parallel. Use auto-scaling: new pods spin up when the queue grows. However, be careful: autoscaling with GPUs can waste resources if jobs are sparse. AI tasks are inherently asynchronous and bursty ²². Synthesia found idle GPU time was a major cost: GPUs were idle over 75% of the time without careful batching ²³. To mitigate this, batch multiple video jobs or scene generations together, or use spot instances. Also implement a timeout/auto-shutdown for idle workers.

In short, use queue-driven scaling (e.g. Kubernetes HPA based on queue length) rather than simple CPU usage. Also consider hybrid modes: reserve a small baseline of GPU capacity for quick jobs, and spin more up on demand. Tools like AWS Batch or ParallelCluster (as Synthesia did ²⁰) can simplify running large inference jobs. The storage layer (S3) should also scale (enable Lifecycle or Glacier for old videos).

Finally, instrument and monitor everything (log progress, GPU utilization). Since video models are probabilistic, implement retry/alternative-model logic if one model fails. Maintain versioning of models for reproducibility.

Front-End Features and UX Design

The UI should guide users through creating a video step-by-step, similar to Synthesia's studio but focused on product videos. Key features and flows:

- **Project Workflow:** Dashboard listing projects. A "New Video" wizard where the user names the video, uploads their product image, and enters a brief prompt or selects a template (e.g. "Tech product ad", "Fashion shoot", etc.). The prompt can be freeform, or assisted by AI suggestions.
- **Scene/Script Editor:** Either allow the user to write a script for each scene, or auto-generate one from the prompt (via a GPT-powered helper). Display this script as editable text, with separate entries for each shot. For example, "Shot 1: [Description of shot], narration: [text]". This helps users plan the video.
- **Asset and Style Selection:** Options to pick voice (gender, language – Synthesia offers 140+ voices ¹⁶), music track, and visual style. For instance, camera motion presets (slow zoom, pan, spin) or filters (cinematic, bright, etc.). Adobe Firefly's image-to-video lets users pre-apply camera moves and effects ²⁴; Evora could offer similar style/camera presets for each scene.
- **Timeline/Storyboard UI:** A visual editor (see image below) where each scene appears as a clip on a timeline. Users can drag to reorder scenes, trim durations, and layer text/graphics. The timeline is a "precision editing tool" – it lets the user control exactly when elements appear ²⁵. For example, text overlays or logos should be movable on the timeline. This mirrors Synthesia's Timeline feature for animation control ²⁵, but here we'd allow ordering of AI-generated clips.



Evora's editor should include a storyboard/timeline view, where users can sequence scenes and layers for precise timing (similar to Synthesia's Timeline feature ²⁵).

- **Preview & Iteration:** After generation, show a low-resolution or streaming preview of the video so users can review. Provide buttons to “Re-generate” with tweaks (e.g. adjust prompt or voice) or “Edit” to change the script or durations. This trial-and-error loop is important; generation may not be perfect on first try.
- **Output and Sharing:** Once satisfied, allow download of the video file. Also integrate sharing (email link, embed code) or direct publishing to social platforms (if possible). Provide multi-format downloads (16:9, 1:1, 9:16) for different channels.
- **User Guidance & Templates:** Include tooltips and example videos. Offer presets like “Product showcase”, “Tutorial”, or “Holiday ad” that pre-fill prompts and styles. A contextual AI assistant (chatbot) could even help refine the prompt or script.
- **Branding Kit:** Let users set brand colors, logos, and fonts upfront (like Synthesia's Brand Kits). The front end should automatically apply these to text overlays and animations to maintain brand consistency.
- **Multi-language Support:** Since marketing videos often go global, support caption generation and dubbing in other languages. For example, translate the script via an LLM and then use a native TTS voice, as Synthesia does with 140+ language options ¹⁶.

Overall, the front-end should feel like a guided video editor: simple enough for non-experts, but with enough controls to customize the output. It will call the backend via REST/WS for actions like “generate storyboard,” “start generation,” and “check status.” Responsive design ensures it works on desktop and tablets. Accessibility (ARIA labels, keyboard nav) is also important for a professional product.

Summary of Recommendations

- **Models:** Use OpenAI Sora 2 Pro (via an API like WaveSpeed) for highest quality image-to-video generation ⁶. Fall back on Stable Video Diffusion or other diffusion models for quicker or cheaper

iterations. Employ an LLM (GPT) to expand prompts into detailed scripts. Use modern TTS (ElevenLabs, AWS Polly) for voice.

- **Backend Tech:** Python for ML workloads, Node.js (or Go) for API services ¹⁸. Containerize and deploy on a cloud GPU cluster (AWS EKS, Azure AKS, or GCP GKE). Use MongoDB (or similar) to store project data JSON ¹⁹; use a relational DB for user/account info. Store media in S3 or equivalent.
- **Pipeline:** Build an asynchronous job queue system where image prompt → scenes → video clip generation → stitching → final encoding. Use Kubernetes/EKS with GPU nodes, and autoscale workers on queue depth (with care to avoid idle GPU costs ²³). Consider AWS Batch or ECS/GPU instances for heavy jobs ²⁰.
- **Frontend:** Create a React-based UI inspired by Synthesia's studio. Features should include image upload, prompt/script editing, voice/music selection, and a **timeline storyboard** (as in Synthesia docs ²⁵). Provide templates, progress indicators, and easy iteration (re-generate, re-edit). Support multiple resolutions and languages.

With this design, Evora.ai can transform a single product photo into a compelling marketing video automatically. By leveraging proven architectures (microservices on cloud GPUs ²⁰), advanced AI models (Sora 2 Pro, Stable Diffusion ⁶ ¹⁴), and an intuitive front-end (scene/script timelines ²⁵), the platform can deliver professional-quality ads at scale. The deep integration of AI at every step – prompt understanding, video generation, voice synthesis – ensures users get “amazingly good” videos with minimal manual effort.

Sources: Modern AI video tools and architectures ³ ⁵ ⁶ ¹⁹ ¹⁸ ²⁵. These references illustrate the technologies (e.g. diffusion models, MongoDB use) and design patterns (polyglot backends, cloud GPU scaling) relevant to building Evora.ai.

¹ ²⁰ ²¹ Synthesia Makes AI Video Production Effortless with Generative AI on AWS
<https://aws.amazon.com/solutions/case-studies/synthesia-case-study/>

² ³ ⁴ Image to Video AI: Top Tools & Build vs Buy Guide 2025
<https://www.tenupsoft.com/blog/image-to-video-ai-enterprise-guide.html>

⁵ ⁷ ⁸ AI for Video Creation: 15 Best Platforms in 2026
<https://monday.com/blog/ai-agents/ai-for-video-creation/>

⁶ ¹⁰ ¹¹ ¹³ ¹⁵ ¹⁷ Sora 2 Pro Image to Video API | WaveSpeedAI
<https://wavespeed.ai/models/openai/sora-2-pro/image-to-video>

⁹ AI Frames to Video Generator: Create Smooth Animations
<https://www.domoai.app/en/create/frames-to-video>

¹² Introducing OpenAI Sora 2 Pro Image-to-Video on WaveSpeedAI | WaveSpeed Blog
<https://wavespeed.ai/blog/posts/introducing-openai-sora-2-pro-image-to-video-on-wavespeedai/>

¹⁴ stabilityai/stable-video-diffusion-img2vid-xt · Hugging Face
<https://huggingface.co/stabilityai/stable-video-diffusion-img2vid-xt>

¹⁶ Synthesia Alternatives for AI-Powered Video Marketing | Vidyard
<https://www.vidyard.com/blog/compare-synthesia-alternatives/>

18 Choosing the Right Tech Stack: Python vs Node.js for AI and Product | Vishal R Gadiya posted on the topic | LinkedIn

https://www.linkedin.com/posts/vishal-gadiya_when-i-was-working-with-a-startup-and-we-activity-7425526757270700032-xGmZ

19 Scalable Architecture for a Real-Time Collaborative Editor

<https://www.synthesia.io/post/scalable-architecture-collaborative-video-editor>

22 23 Cost-Efficient Autoscaling Strategies for AI Workloads

<https://www.cloudoptimo.com/blog/cost-efficient-autoscaling-strategies-for-ai-workloads/>

24 Image to video AI - Adobe Firefly

<https://www.adobe.com/products/firefly/features/image-to-video.html>

25 Timeline

<https://docs.synthesia.io/docs/timeline>